FinWise — Guion para 5 expositores (8 minutos y 10 segundos)

Presentación principal: diapositivas 1 a 7. **Diapositiva 8:** anexo DER, únicamente si hay preguntas. **Duración objetivo:** 8:10; margen hasta máximo 9:00: 50 segundos. Los tiempos son orientativos y deben confirmarse ensayando en voz alta.

Integrante 1 — 0:00 a 1:30 (diapositivas 1 y 2)

Texto para decir (aprox. 1:30)

"Buenos días. Somos el equipo de FinWise, una aplicación de gestión financiera personal. El propósito es que una persona pueda registrar su dinero, organizar sus compromisos y tomar decisiones con información clara. Para lograrlo diseñamos una base de datos relacional en PostgreSQL, usando Supabase para persistencia y autenticación, mientras que Next.js será la interfaz y la capa de interacción de la aplicación.

Nuestro modelo general tiene catorce entidades: una gestionada por Supabase Auth y trece propias de FinWise. No son tablas elegidas por cantidad: cada una responde a funciones previstas y a los casos de uso documentados.

Las necesidades se agrupan en tres: registrar cuentas, ingresos y gastos; planificar obligaciones, presupuestos y ahorros; y analizar deudas, cobros periódicos y alertas. Para cada función asociamos entidades, relaciones y condiciones de prueba. Ahora veremos cómo se organiza el modelo."

Idea clave: explicar el problema y la trazabilidad, no enumerar las 14 tablas.

Integrante 2 — 1:30 a 3:00 (diapositiva 3)

Texto para decir (aprox. 1:30)

"El primer bloque contiene cinco entidades. `auth.users` se encarga de identificar y autenticar a cada persona. `profiles`, relacionado uno a uno con el usuario, guarda preferencias como la moneda y una reserva de seguridad. `accounts` almacena las cuentas financieras del usuario, por ejemplo efectivo o una cuenta bancaria. `categories` clasifica los movimientos, como alimentación o salario. Finalmente, `transactions` registra ingresos, gastos y transferencias, con su importe, fecha y cuenta correspondiente.

Las relaciones son fundamentales: un usuario puede tener muchas cuentas y una cuenta puede registrar muchos movimientos. Asimismo, una categoría clasifica varios movimientos. Esto permite responder preguntas como cuánto dinero tengo, cuánto recibí durante el mes y en qué categoría estoy gastando más.

Como regla del sistema, cada movimiento debe pertenecer al usuario autenticado. El saldo de una cuenta puede calcularse desde su saldo inicial y sus movimientos,

sin duplicar innecesariamente esa información."

Ejemplo: cuenta con 500.000 COP iniciales + ingreso de 200.000 COP – gasto de 50.000 COP = saldo de 650.000 COP.

Integrante 3 — 3:00 a 4:45 (diapositiva 4)

Texto para decir (aprox. 1:45)

"El segundo bloque se enfoca en la planeación y en las operaciones repetitivas. `obligations` registra compromisos de pago pendientes, por ejemplo una matrícula. `budgets` determina un límite de gasto por categoría durante un periodo. Al relacionar presupuestos con categorías y movimientos, la aplicación puede calcular cuánto se ha consumido del límite establecido.

Para el ahorro, `goals` almacena cada objetivo: por ejemplo reunir un millón de pesos. `goal_contributions` registra los aportes que se realizan. La relación uno a muchos permite sumar esos aportes y conocer el progreso real de la meta.

También se contemplan `recurring_transactions`, para representar movimientos que se repiten con cierta frecuencia, y `subscriptions`, para servicios periódicos como plataformas digitales. Estas tablas permiten anticipar próximos cobros y consultar su impacto en las finanzas.

Por ejemplo, si una persona tiene un presupuesto de trescientos mil pesos para alimentación y ha registrado doscientos mil en gastos de esa categoría, FinWise podrá indicar que ha consumido aproximadamente el 67 % del presupuesto."

No confundir: una suscripción describe un servicio; una recurrencia describe la regla que permite repetir un movimiento.

Integrante 4 — 4:45 a 6:05 (diapositiva 5)

Texto para decir (aprox. 1:20)

"El tercer bloque completa las catorce entidades. `debts` representa las deudas que tiene la persona, su estado y saldo. `debt_payments` registra los pagos realizados y permite distinguir componentes como capital e intereses. Su relación es uno a muchos: una deuda puede recibir diferentes pagos a lo largo del tiempo.

La entidad `alerts` registra los avisos que verá el usuario, por ejemplo un compromiso próximo a vencer o una advertencia financiera, y permite identificar cuáles ya fueron leídos.

El diagrama entidad–relación une estos bloques a través de claves primarias y foráneas. Por ejemplo, una categoría se conecta con transacciones y presupuestos; una meta se conecta con sus aportes; y una deuda con sus pagos.

Así, los datos no quedan aislados: cada relación responde a preguntas que la aplicación debe resolver. El diagrama completo está incluido como anexo por si desean revisar todas las conexiones."

Idea clave: hablar de cardinalidad y de consultas, no solo de nombres de tablas.

Integrante 5 — 6:05 a 8:10 (diapositivas 6 y 7)

Texto para decir (aprox. 2:05)

"Finalmente, convertimos el modelo conceptual en un esquema técnico para PostgreSQL, desplegable en Supabase. Las tablas se identifican con claves primarias UUID y se relacionan mediante claves foráneas. Para manejar dinero proponemos `numeric(20,4)`, que evita los errores de redondeo de los tipos de punto flotante. También definimos reglas de integridad, como impedir importes negativos cuando no tengan sentido, y restricciones para evitar registros inconsistentes.

La seguridad es esencial porque hablamos de información financiera. Por eso el diseño incluye políticas de seguridad a nivel de fila, conocidas como RLS, cuyo objetivo es que cada usuario consulte y modifique únicamente sus propios datos. Además, los indicadores del panel de control se calcularán a partir de las tablas del modelo; no hace falta almacenar una tabla independiente llamada dashboard.

Para demostrar que el software cumple los requisitos, proponemos tres pruebas de aceptación. Primera: registrar un gasto y verificar que se guarde y disminuya el saldo calculado. Segunda: registrar gastos de una categoría y comprobar que el porcentaje del presupuesto se actualice. Tercera: intentar consultar datos de otro usuario y verificar que la base de datos deniegue el acceso.

Las pruebas están especificadas y deberán ejecutarse durante la implementación. En conclusión, el diseño de FinWise vincula necesidades del usuario, estructura de datos y verificación funcional, sin agregar entidades innecesarias."

Cuidado: no decir que las pruebas pasaron o que todas las funcionalidades ya están desarrolladas.

Matriz de pruebas para mostrar si el profesor pregunta

ID	Especificación	Datos de entrada	Acción desde Next.js	Resultado esperado	Capas
T-01	Los movimientos actualizan el saldo calculado	Saldo inicial 500.000; gasto 50.000	Enviar formulario de gasto	Movimiento almacenado; saldo calculado 450.000	UI + acción servidor + BD

ID	Especificación	Datos de entrada	Acción desde Next.js	Resultado esperado	Capas
T-02	Los presupuestos reflejan consumo por categoría	Presupuesto 300.000; gastos 200.000	Consultar vista de presupuesto	Consumido 66,67%; disponible 100.000	Consulta agregada + vista
T-03	Los datos de un usuario son privados	Usuarios A y B; cuenta de B	A consulta identificador de cuenta de B	Consulta sin acceso a información de B	Supabase RLS
T-04	Aportes actualizan progreso de meta	Meta 1.000.000; aportes 250.000 y 150.000	Consultar detalle de meta	Ahorro acumulado 400.000; progreso 40 %	BD + cálculo + UI
T-05	Los pagos pertenecen a una deuda	Deuda con dos pagos válidos	Guardar pagos	Dos filas asociadas; trazabilidad por deuda	FK + integración
T-06	Los montos no admiten valores inválidos	Importe negativo en un campo que requiere valor positivo	Enviar formulario	Error de validación; sin fila inválida	Formulario + SQL CHECK

Estos casos son **propuestos, no ejecutados**. Para pruebas de integración, usar una base de datos de desarrollo con usuarios de prueba.

Preguntas probables y respuestas cortas

¿Por qué 14 entidades? Porque el conjunto cubre los módulos previstos en el diseño general: identidad, dinero, planificación, ahorro, recurrencias, deudas y alertas. Cada entidad tiene una responsabilidad distinta.

¿Qué significa 1:N? Que un registro del origen puede relacionarse con muchos registros del destino; por ejemplo una meta con muchos aportes.

¿Qué diferencia hay entre PK y FK? La PK identifica una fila de su tabla; la FK referencia una fila existente en otra tabla y protege la relación.

¿Por qué no hay tabla dashboard? Porque un dashboard es una vista que se construye con cálculos y consultas a datos transaccionales.

¿Qué es RLS? Una regla de seguridad de PostgreSQL que limita las filas a las que puede acceder cada usuario según su identidad.

¿Están implementadas las 14 tablas? Se presentan como diseño previsto de la aplicación integral. La exposición no afirma que todas estén desplegadas ni probadas.

Estrategia de ensayo

1. Ensayar una vez por persona con cronómetro; recortar ejemplos si alguien supera su bloque.
2. Ensayar el relevo entre personas sin presentaciones largas: bastará una frase de transición.
3. Ensayar todos juntos y terminar idealmente entre 8:00 y 8:20 para mantener margen ante pausas.
4. Dejar la diapositiva 8 exclusivamente para preguntas; no consumir tiempo principal en ella.